

AI4Trade Bot — Audit- und GAP-Analyse

2026-05-29

Projektname: ai4trade-bot

Kontext: Python-basierter Krypto-Signalgenerator fuer AI4Trade im dry_run-Modus mit Bitget-Marktdaten, technischer Analyse, KI-Sentiment und SQLite-gestuetzter Persistenz.

Stichtag: 2026-05-29

Datenerfassung: 2026-05-29T15:14:38.4240164+02:00 bis 2026-05-29T15:14:57.8888741+02:00 sowie vollstaendiger Testlauf am 2026-05-29 (320 passed in 57.19s).

Audit-Typ: Repository-basierter Projekt- und Reifegradaudit ohne Produktionszugriff.

Referenzrahmen: ISO/IEC 27001:2022 Annex A, NIST SSDF 1.1, ITIL 4, NIST SP 800-61 Rev. 2.

1. Executive Summary

Der aktuelle Stand von ai4trade-bot ist fuer ein MVP technisch solide: Die Codebasis ist modular, die lokale Testlage ist stark (320 erfolgreiche Tests), eine CI-Pipeline ist vorhanden und zentrale Kernfunktionen wie Signalgenerierung, Publishing, SQLite-Persistenz und Basis-Betriebsdokumentation sind nachweislich implementiert.

Gleichzeitig zeigt der Audit eine deutliche Luecke zwischen vorhandenen Bausteinen und tatsaechlich aktiviertem Betriebsverhalten. Besonders kritisch ist, dass Health- und Metrics-Funktionalitaet zwar als Python-Module vorliegt, aber im aktiven Laufzeitpfad nicht exponiert wird, waehrend Docker-Healthcheck, Prometheus-Scraping und Betriebsdoku genau diese Endpunkte voraussetzen. Zusaetzlich ist der Observability-Stack fehlverdrahtet, das Dependency-Lockfile driftet gegen die deklarativen Eingaben, und mehrere Sicherheits- bzw. Safety-Komponenten sind implementiert und getestet, aber nicht im Default-Laufzeitpfad verdrahtet.

Gesamturteil:

- Bereit fuer beaufsichtigten Entwickler- und CI-gestuetzten dry_run-Betrieb.
- Nicht bereit fuer unbeaufsichtigten, containerisierten Dauerbetrieb gemaess aktueller Doku.
- Nicht bereit fuer jede Form von Live-Aktivierung oder regulatorisch anspruchsvollen Einsatz.

Top-Feststellungen:

- Staerken: modulare Architektur, CI vorhanden, starke Testbasis, SQLite mit WAL, Secret-Provider, Incident- und Recovery-Doku.
- Kritische Luecken: kein aktiver /health-/metrics-Endpoint, Observability-Stack fehlverdrahtet, Requirements-Drift, fehlendes Default-Rate-Limiting, unvollstaendig aktivierte Safety-Kontrollen.
- Reifegrad: gute Engineering-Basis, aber nur teilweise operationalisiert.

2. Projektumfang und -ziele

In Scope dieses Audits:

- Python-Code im Repository inklusive main.py, core/, adapters/, exchanges/, execution/, trading/, storage/, ai/, chat/.

- Build- und Betriebsartefakte: pyproject.toml, requirements*, Dockerfile, docker-compose.yml, .github/workflows/ci.yml, monitoring/, scripts/backup.sh.
- Dokumentation: README.md, docs/operations.md, docs/incident-response.md, docs/recovery.md und vorhandene Audit-Artefakte.
- Testlage: tests/ plus ausgefuehrter Volltestlauf.

Nicht in Scope:

- Produktivsysteme, laufende Container, externe APIs, reale AI4Trade- oder Exchange-Accounts.
- Interviews mit Produkt-, Ops-, Security- oder Compliance-Verantwortlichen.
- Rechtsberatung zu Markt- oder Finanzregulierung.

Audit-Ziele:

- Tatsaechlichen Implementierungsstand gegen dokumentierte Soll-Annahmen abgleichen.
- Aktivierte Kontrollen von nur vorhandenen, aber nicht angeschlossenen Kontrollen unterscheiden.
- Betriebs-, Sicherheits-, Traceability- und Governance-Luecken priorisieren.
- Eine belastbare Entscheidungshilfe fuer die Bereitschaft des Projekts liefern.

Verwendete Referenzrahmen:

- ISO/IEC 27001:2022 Annex A fuer dokumentierte Verfahren, Monitoring, sichere Konfiguration, Kryptographie und operative Kontrollen.
- NIST SSDF 1.1 fuer Build-/Dependency-Disziplin, sichere Software-Lieferung und Security-Gates.
- ITIL 4 fuer Monitoring, Event Management, Change-/Configuration-Management und Service Continuity.
- NIST SP 800-61 Rev. 2 fuer Incident-Response- und Recoverability-Betrachtung.

Wichtig: Dies ist kein Zertifizierungsaudit. Die Standards dienen als praxisorientierter Referenzrahmen fuer eine technische GAP-Analyse.

3. Audit-Methodik

Die Bewertung basiert auf vier komplementaeren Methoden:

- Artefaktpruefung: Quellcode, Konfigurationsdateien, Betriebsdokumentation, Monitoring-Konfiguration, CI-Workflow und Backup-Skript wurden direkt gelesen.
- Laufzeitpfad-Analyse: Es wurde gezielt geprueft, welche Komponenten von main.py und den Default-Factories tatsaechlich instanziiert und genutzt werden.
- Automatisierte Verifikation: get_errors meldete keine aktuellen Editor-/Analysefehler; der Volltestlauf lieferte 320 passed in 57.19s.
- Konsistenzpruefung: Dokumentation, deklarative Requirements, kompilierten Requirements, Docker-/Monitoring-Definitionen und aktive Codepfade wurden gegeneinander abgeglichen.

Datenquellen und Belege:

- Code: main.py, config.py, core/health.py, core/metrics.py, adapters/signal_publisher.py, exchanges/factory.py, exchanges/bitget_rest.py, adapters/rate_limiter.py, trading/safety_gateway.py, trading/portfolio_circuit_breaker.py, storage/sqlite_repository.py.

- Betrieb: Dockerfile, docker-compose.yml, monitoring/prometheus.yml, monitoring/alertmanager_rules.yml, scripts/backup.sh.
- Governance und Doku: README.md, CHANGELOG.md, docs/operations.md, docs/incident-response.md, docs/recovery.md, .github/workflows/ci.yml.
- Testbelege: tests/ sowie der ausgefuehrte Pytest-Lauf.

Interview- und Annahmenmodell:

- Es wurden keine Interviews durchgefuehrt.
- Aussagen zu Betriebsrealitaet, Restore-Drills, On-Call-Prozessen oder Produktiv-Telemetrie werden deshalb explizit als Annahmen oder Datenluecken markiert.

Audit-Einschraenkungen:

- Keine Validierung gegen Live-Systeme.
- Keine Lasttests oder Performance-Messungen im Produktivbetrieb.
- Keine Einsicht in die reale .env-Belegung; nur Vorhandensein und Integrationspfade wurden betrachtet.

4. Bewertung des aktuellen Zustands

4.1 Nachweislich implementierte und funktionierende Kernbausteine

- Qualitaetssicherung:
- .github/workflows/ci.yml fuehrt Linting, Tests und Security-Scans aus.
- Der lokale Volltestlauf am 2026-05-29 war erfolgreich: 320 passed in 57.19s.
- get_errors meldete fuer das Workspace-Root keine aktuellen Fehler.
- Kernlaufzeit:
- main.py instanziiert MarketData, TechnicalAnalyzer, SentimentAnalyzer, Strategy, RiskGate, SignalRouter, Heartbeat, TaskHandler sowie SqliteSignalRepository und SignalPublisher.
- exchanges/factory.py erzeugt standardmaessig einen BitgetRestClient; core/market_data.py nutzt CoinGecko als Fallback fuer OHLCV.
- Persistenz und Audit:
- storage/sqlite_repository.py initialisiert SQLite im WAL-Modus und legt signals, app_state und audit_log an.
- main.py schreibt bot_start und bot_stop in das Audit-Log.
- adapters/signal_publisher.py persistiert erfolgreich veroeffentlichte Signale.
- Sicherheits- und Support-Bausteine:
- core/secret_provider.py bietet env, keyring und vault Backends.
- core/ssl_context.py, core/two_factor.py, core/health.py und core/metrics.py sind vorhanden und getestet.
- Betriebsdokumentation:
- docs/operations.md, docs/incident-response.md und docs/recovery.md dokumentieren Start/Stop, Incident Response und Backup/Recovery.

4.2 Vorhanden, aber nur teilweise operationalisiert

- Observability:

- core/health.py und core/metrics.py liefern Helper-Funktionalitaet, aber es wurde kein HTTP-Server gefunden, der /health oder /metrics exponiert.
- config.METRICS_PORT ist konfigurierbar, wird im aktiven Laufzeitpfad jedoch nicht verwendet.
- Safety-/Execution-Layer:
- trading/safety_gateway.py, trading/portfolio_circuit_breaker.py, execution/order_executor.py und execution/shadow_executor.py sind implementiert und getestet.
- main.py nutzt diese Bausteine jedoch nicht im Default-Pfad.
- Rate Limiting:
- adapters/rate_limiter.py und BitgetRestClient(rate_limiter=...) existieren.
- exchanges/factory.py injiziert im Default-Fall keinen Rate Limiter.
- Future/Partial Features:
- adapters/task_handler.py drainiert nur Queue-Eintraege und loggt sie.
- integrations/freqtrade_bridge.py ist ein Platzhalter.
- exchanges/market_stream.py bietet einen NoOpMarketStream; der Bitget-WebSocket-Pfad ist NotImplemented.

4.3 Gesammelte Belege pro Hauptkomponente

- Laufzeit-Orchestrierung: main.py
- Konfiguration: config.py, config_schema.py, .env, .env.example
- Daten und Analyse: core/market_data.py, core/technical.py, core/strategy.py, ai/providers/, core/llm.py
- Publishing und Persistenz: adapters/signal_publisher.py, storage/sqlite_repository.py
- Ops und Plattform: Dockerfile, docker-compose.yml, monitoring/prometheus.yml, monitoring/alertmanager_rules.yml, scripts/backup.sh
- Security und Kontrollen: core/secret_provider.py, core/two_factor.py, core/ssl_context.py, trading/safety_gateway.py, trading/portfolio_circuit_breaker.py
- Testnachweis: tests/ und lokaler Lauf 320 passed in 57.19s

5. GAP-Analyse

Die GAP-Analyse vergleicht den beobachteten Ist-Zustand mit vier Sollperspektiven:

- Operability-Soll: Dokumentierte Betriebs- und Monitoring-Funktionen muessen im Default-Laufzeitpfad aktiv und verifizierbar sein.
- Secure-Delivery-Soll: Deklarative und kompiliert ausgelieferte Abhaengigkeiten, Security-Gates und Kontrollen muessen konsistent sein.
- Runtime-Control-Soll: Implementierte Sicherheits- und Safety-Bausteine muessen entweder bewusst aktiviert oder bewusst als ausserhalb des Scopes markiert sein.
- Traceability-Soll: Der Signalpfad muss nachvollziehbar dokumentier- und auditierbar sein, inklusive Retry-, Block- und Nebenpfaden.

Ergebnis:

- Der Code erfuellt das Entwicklungs-Soll deutlich besser als das Betriebs-Soll.
- Die staerksten Abweichungen liegen nicht in fehlender Fachlogik, sondern in fehlender Verdrahtung, inkonsistenter Konfiguration und veralteter Dokumentation.

- Mehrere Kontrollen existieren bereits, liefern aber im Default-Laufzeitpfad noch keinen belastbaren Nutzen.

6. Identifizierte Luecken

GAP-01 — Health- und Metrics-Endpunkte fehlen im aktiven Laufzeitpfad

- Referenz: ISO/IEC 27001:2022 Annex A.8.16, ITIL 4 Monitoring and Event Management.
- Aktueller Status: Offen; Helper-Module vorhanden, aber im Default-Betrieb nicht exponiert.
- Risikostufe / Schweregrad / Prioritaet: Kritisch / Hoch / P1.
- Beobachtete Symptome:
- core/health.py und core/metrics.py existieren.
- main.py startet keinen HTTP-Server und nutzt METRICS_PORT nicht.
- docker-compose.yml und Dockerfile erwarten http://localhost:9090/health.
- Auswirkung: Container-Healthchecks, Prometheus-Scraping und dokumentierte Betriebschecks koennen nicht wie beschrieben funktionieren.
- Belege: main.py, config.py, core/health.py, core/metrics.py, Dockerfile, docker-compose.yml, monitoring/prometheus.yml.

GAP-02 — Observability-Stack ist fehlverdrahtet und inkonsistent

- Referenz: ISO/IEC 27001:2022 Annex A.8.16, ITIL 4 Event Management.
- Aktueller Status: Offen; Stack definiert, aber nicht konsistent anschlussfaehig.
- Risikostufe / Schweregrad / Prioritaet: Kritisch / Hoch / P1.
- Beobachtete Symptome:
- monitoring/prometheus.yml referenziert alertmanager_rules.yml, aber die Datei wird dem Prometheus-Container nicht gemountet.
- docker-compose.yml bindet monitoring/alertmanager_rules.yml stattdessen als Alertmanager-config.yml ein.
- monitoring/alertmanager_rules.yml verwendet ai4trade_*-Metriknamen, waehrend core/metrics.py bot_*-Metriken definiert.
- Auswirkung: Selbst bei spaeterer Endpoint-Exposition blieben Regeln und Alerts in der aktuellen Form unzuverlaessig oder funktionslos.
- Belege: docker-compose.yml, monitoring/prometheus.yml, monitoring/alertmanager_rules.yml, core/metrics.py.

GAP-03 — Deklarative und kompiliert ausgelieferte Requirements driften auseinander

- Referenz: NIST SSDF 1.1 PW.4, ISO/IEC 27001:2022 Annex A.8.9.
- Aktueller Status: Offen; requirements.in und requirements*.txt sind nicht konsistent.
- Risikostufe / Schweregrad / Prioritaet: Hoch / Hoch / P1.
- Beobachtete Symptome:
- requirements.in listet prometheus_client und pyotp.
- requirements.txt und requirements-dev.txt enthalten diese Pakete nicht.
- Der lokale Testlauf war erfolgreich, was auf eine vom Lockfile abweichende Umgebung hindeutet.

- Auswirkung: Clean Builds, Docker-Images und CI-Installationen koennen vom lokal getesteten Zustand abweichen.
- Belege: requirements.in, requirements.txt, requirements-dev.txt, tests/test_metrics.py, tests/test_two_factor.py.

GAP-04 — Dokumentation und Versionsangaben sind nicht mehr Source of Truth

- Referenz: ISO/IEC 27001:2022 Annex A.5.37, NIST SSDF 1.1 PO.3.
- Aktueller Status: Offen; mehrere zentrale Dokumente sind veraltet oder widerspruechlich.
- Risikostufe / Schweregrad / Prioritaet: Hoch / Mittel / P2.
- Beobachtete Symptome:
- README.md nennt Python 3.10+, pyproject.toml fordert >=3.11.
- README.md, CHANGELOG.md und vorhandene Audit-Artefakte nennen alte Teststaende und Versionen.
- docs/operations.md beschreibt Endpunkte und Monitoring-Annahmen, die im aktiven Pfad nicht nachweisbar sind.
- Auswirkung: Onboarding, Deployments und Management-Reporting koennen auf falschen Annahmen basieren.
- Belege: README.md, CHANGELOG.md, pyproject.toml, docs/operations.md, bestehende Dateien unter docs/audit/.

GAP-05 — Default-Laufzeitpfad verwendet kein Rate Limiting

- Referenz: ISO/IEC 27001:2022 Annex A.8.16, ITIL 4 Availability and Capacity Management.
- Aktueller Status: Offen; Rate-Limiter existiert, ist aber im Default-Fabrikpfad nicht aktiv.
- Risikostufe / Schweregrad / Prioritaet: Hoch / Hoch / P1.
- Beobachtete Symptome:
- adapters/rate_limiter.py implementiert einen Token-Bucket.
- exchanges/bitget_rest.py kann einen Rate Limiter nutzen.
- exchanges/factory.py erstellt BitgetRestClient() ohne Rate Limiter; main.py uebergibt keinen.
- Auswirkung: Die dokumentierten RATE_LIMIT_*-Werte sind im Normalpfad wirkungslos; API-Drosselung oder instabiles Verhalten bleiben moeglich.
- Belege: config.py, adapters/rate_limiter.py, exchanges/bitget_rest.py, exchanges/factory.py, main.py.

GAP-06 — Zusaeztliche Safety- und Execution-Kontrollen sind nicht im Main-Pfad verdrahtet

- Referenz: ISO/IEC 27001:2022 Annex A.8.28 und A.8.32, NIST SSDF 1.1 PW.5.
- Aktueller Status: Offen; implementiert und getestet, aber nicht aktiv im Default-Betrieb.
- Risikostufe / Schweregrad / Prioritaet: Hoch / Hoch / P1.
- Beobachtete Symptome:
- main.py nutzt RiskGate und SignalRouter, aber nicht SafetyGateway, PortfolioCircuitBreaker, OrderExecutor oder ShadowExecutor.
- Umfangreiche Tests in tests/test_safety_gateway.py, tests/test_portfolio_circuit_breaker.py, tests/execution/ zeigen, dass diese Bausteine als relevant betrachtet wurden.

- Auswirkung: Zwischen getesteter Kontrolllandschaft und tatsächlicher Produktionsverdrahtung besteht eine gefährliche Lücke.
- Belege: main.py, trading/safety_gateway.py, trading/portfolio_circuit_breaker.py, execution/order_executor.py, execution/shadow_executor.py, zugehörige Tests.

GAP-07 — 2FA ist latent und im Fehlerfall fail-open

- Referenz: ISO/IEC 27001:2022 Annex A.5.17 und A.5.18.
- Aktueller Status: Offen; für spätere Live-Fähigkeit relevant, aktuell nicht sauber operationalisiert.
- Risikostufe / Schweregrad / Priorität: Hoch / Mittel / P3.
- Beobachtete Symptome:
- core/two_factor.py gibt True zurück, wenn kein TOTP-Secret konfiguriert ist.
- main.py führt die 2FA-Prüfung nur im Nicht-dry_run-Pfad aus und beendet anschließend ohnehin mit Nur dry_run ist unterstützt.
- Auswirkung: Die Kontrolle ist aktuell weder produktiv wirksam noch für einen künftigen Live-Pfad ausreichend häftet.
- Belege: core/two_factor.py, main.py, tests/test_two_factor.py.

GAP-08 — Certificate Pinning ist standardmässig wirkungslos und nicht erzwungen

- Referenz: ISO/IEC 27001:2022 Annex A.8.24, NIST SSDF 1.1 PS.2.
- Aktueller Status: Offen; Modul vorhanden, Default-Konfiguration leer.
- Risikostufe / Schweregrad / Priorität: Mittel / Mittel / P3.
- Beobachtete Symptome:
- core/ssl_context.py startet mit leerem KNOWN_FINGERPRINTS-Mapping.
- Ohne konfigurierten Pin gibt verify_fingerprint() standardmässig True zurück.
- Im gelesenen Default-Laufzeitpfad wird kein aktiver Pinning-Einsatz erzwungen.
- Auswirkung: Das Modul vermittelt eine vorhandene Sicherheitsfähigkeit, die in der Standardkonfiguration keinen Schutzbeitrag liefert.
- Belege: core/ssl_context.py, tests/test_ssl_context.py, exchanges/bitget_rest.py.

GAP-09 — Teilimplementierte Nebenpfade und Platzhalter sind nicht klar abgegrenzt

- Referenz: ITIL 4 Service Design, NIST SSDF 1.1 PO.3.
- Aktueller Status: Offen; mehrere Features sind sichtbar, aber nicht betriebsreif.
- Risikostufe / Schweregrad / Priorität: Mittel / Mittel / P3.
- Beobachtete Symptome:
- adapters/task_handler.py loggt Nachrichten, führt aber keine echte Tasklogik aus.
- integrations/freqtrade_bridge.py ist ein Platzhalter.
- exchanges/market_stream.py nutzt im Default einen NoOpMarketStream; der Bitget-WebSocket-Pfad ist NotImplemented.
- Auswirkung: Scope und Reifegrad dieser Features sind für Leserinnen, Leser und Betreiber nicht eindeutig.

- Belege: adapters/task_handler.py, integrations/freqtrade_bridge.py, exchanges/market_stream.py.

GAP-10 — Backup- und Recovery-Faehigkeit ist nicht end-to-end validiert

- Referenz: ISO/IEC 27001:2022 Annex A.5.30, ITIL 4 Service Continuity, NIST SP 800-61 Rev. 2.
- Aktueller Status: Teilweise; Doku und Skript vorhanden, Betriebswirksamkeit nicht nachgewiesen.
- Risikostufe / Schweregrad / Prioritaet: Hoch / Mittel / P2.
- Beobachtete Symptome:
- scripts/backup.sh und docs/recovery.md definieren ein Backup-/Restore-Vorgehen.
- Ein reproduzierter Restore-Drill oder CI-validierter Recovery-Test ist im Repository nicht erkennbar.
- Dockerfile provisioniert benoetigte Kommandozeilenwerkzeuge fuer Healthcheck/Backup nicht explizit; deren Verfuegbarkeit im Zielimage wurde in diesem Audit nicht bestaetigt.
- Auswirkung: Die dokumentierte Wiederanlaufstrategie ist ohne Durchstichstest nicht belastbar.
- Belege: scripts/backup.sh, docs/recovery.md, docs/operations.md, Dockerfile.
- Annahme: Die Tool-Verfuegbarkeit im finalen Runtime-Image wurde nicht extern validiert.

GAP-11 — Audit-Trail ist nur teilweise vollstaendig

- Referenz: ISO/IEC 27001:2022 Annex A.8.15 und A.8.16.
- Aktueller Status: Teilweise; Signale und Bot-Start/Stop werden erfasst, der End-to-End-Entscheidungspfad nicht.
- Risikostufe / Schweregrad / Prioritaet: Hoch / Mittel / P2.
- Beobachtete Symptome:
- SignalPublisher.publish() speichert nur erfolgreich veroeffentlichte Signale.
- flush_queue() sendet erfolgreich erneut, persistiert diese Erfolge aber nicht.
- Blockierte Signale, RiskGate-Entscheidungen und sonstige Kontrollentscheidungen werden im Standardpfad nicht systematisch persistiert.
- Auswirkung: Nachvollziehbarkeit fuer Retry-, Block- und Kontrollpfade bleibt unvollstaendig.
- Belege: adapters/signal_publisher.py, storage/sqlite_repository.py, main.py, execution/execution_audit.py.

GAP-12 — Security-Gates in CI sind nur advisory

- Referenz: NIST SSDF 1.1 PW.7 und RV.1.
- Aktueller Status: Offen; Security-Job vorhanden, aber nicht blockierend.
- Risikostufe / Schweregrad / Prioritaet: Mittel / Mittel / P2.
- Beobachtete Symptome:
- .github/workflows/ci.yml enthaelt pip-audit und bandit.
- Der security-Job ist continue-on-error: true.
- Container-Scanning und Secret-Scanning wurden im gelesenen Workflow nicht gefunden.
- Auswirkung: Sicherheitsbefunde koennen in die Hauptlinie gelangen, ohne Releases technisch zu blockieren.
- Belege: .github/workflows/ci.yml.

7. Auswirkungs- und Risikobewertung

Priorisierte Risikorangfolge:

- Rang 1: GAP-01 — Ohne aktive Health-/Metrics-Endpunkte ist der dokumentierte Betriebsmodus nicht belastbar.
- Rang 2: GAP-02 — Der Observability-Stack ist aktuell nicht konsistent anschlussfaehig.
- Rang 3: GAP-05 — Default-Betrieb ohne Rate Limiting erzeugt direkte API- und Stabilitaetsrisiken.
- Rang 4: GAP-03 — Requirements-Drift untergraebt Reproduzierbarkeit und Lieferqualitaet.
- Rang 5: GAP-06 — Nicht verdrahtete Safety-Kontrollen erzeugen ein falsches Sicherheitsgefuehl.
- Rang 6: GAP-11 — Teilweise Auditierbarkeit schwaecht Root-Cause-Analyse und Governance.
- Rang 7: GAP-10 — Wiederanlaufprozesse sind dokumentiert, aber nicht nachweisbar gehaertet.
- Rang 8: GAP-07 — 2FA ist fuer kuenftige Aktivierung nicht betriebssicher.
- Rang 9: GAP-12 — Security-Scans sind informativ, aber nicht gate-wirksam.
- Rang 10: GAP-04 — Dokumentationsdrift schafft operative Fehlannahmen.
- Rang 11: GAP-08 — Certificate Pinning ist aktuell eher ein Versprechen als ein Schutzmechanismus.
- Rang 12: GAP-09 — Teilimplementierungen vernebeln Scope und Betriebsreife.

Risikowirkungen nach Kategorie:

- Betrieb: Beobachtungsblindheit, fehlerhafte Healthchecks, unklare Wiederanlaufrealitaet.
- Sicherheit: Nicht aktivierte oder fail-open Kontrollpfade, nicht blockierende Security-Scans.
- Governance: Unterschied zwischen Dokumentation, Lieferartefakten und Realitaet.
- Compliance/Traceability: Teilweise Audit-Spur, fehlende Vollstaendigkeit in Nebenpfaden.

8. Empfehlungen und Abhilfemassnahmen

Welle 1 — bis 2026-06-12:

- GAP-01 und GAP-02
- Schritte: HTTP-Exposure fuer /health und /metrics implementieren; METRICS_PORT aktiv verwenden; Prometheus- und Alertmanager-Dateien korrekt mounten; Metriknamen zwischen Code und Regeln harmonisieren.
- Eigentuerer: Backend + DevOps.
- Ressourcen: 2 bis 3 Engineering-Tage.
- Erfolgskriterien: docker compose-Healthcheck ist gruen; Prometheus scrapet Metriken; Alerting-Regeln werden geladen und referenzieren existierende Metriken.
- GAP-03
- Schritte: requirements.txt und requirements-dev.txt aus requirements*.in neu generieren; Clean-Venv- und Docker-Install verifizieren.
- Eigentuerer: Backend / Build-Verantwortung.
- Ressourcen: 0.5 bis 1 Tag.
- Erfolgskriterien: reproduzierbare Neuinstallation ohne lokale Sonderpakete; CI und lokaler Testlauf verhalten sich konsistent.
- GAP-05

- Schritte: Rate Limiter im Default-Factory-Pfad instanziiieren; Konfigurationswerte RATE_LIMIT_* an echte Laufzeit koppeln.
- Eigentuerer: Backend.
- Ressourcen: 1 Tag.
- Erfolgskriterien: Default-Bitget-Pfad nutzt Rate Limiting nachweisbar; Tests decken den Wiring-Pfad ab.
- GAP-04
- Schritte: README, Changelog und Ops-Doku auf Versions-, Test- und Exchange-Realitaet synchronisieren.
- Eigentuerer: Tech Lead + Maintainer.
- Ressourcen: 0.5 bis 1 Tag.
- Erfolgskriterien: eine konsistente, aktuelle Dokumentationslinie ohne bekannte Widersprueche.

Welle 2 — bis 2026-06-26:

- GAP-06
- Schritte: Entscheidung treffen, ob SafetyGateway, PortfolioCircuitBreaker und Execution-Pfade in den Main-Pfad gehoeren oder formal ausserhalb des MVP-Scope liegen; entsprechend integrieren oder explizit stilllegen.
- Eigentuerer: Tech Lead + Backend.
- Ressourcen: 2 bis 3 Tage.
- Erfolgskriterien: kein bedeutender Kontrollbaustein verbleibt unentschieden zwischen „getestet“ und „nicht aktiv“.
- GAP-10
- Schritte: Restore-Drill auf sauberer Umgebung durchfuehren; benoetigte Runtime-Tools validieren; dokumentierte RTO/RPO fuer den aktuellen dry-run Scope festhalten.
- Eigentuerer: DevOps / Ops.
- Ressourcen: 1 bis 2 Tage.
- Erfolgskriterien: wiederholbarer Restore-Nachweis und aktualisierte Runbooks.
- GAP-11
- Schritte: Audit-Trail um Queue-Flush, Blockentscheidungen, Kontrollresultate und Korrelations-IDs erweitern.
- Eigentuerer: Backend + Compliance/Tech Lead.
- Ressourcen: 2 Tage.
- Erfolgskriterien: jeder relevante Signalpfad ist durchgaengig nachvollziehbar.
- GAP-12
- Schritte: Security-Scans blockierend machen oder explizite Risikoakzeptanz dokumentieren; Secret- und Container-Scanning ergaenzen.
- Eigentuerer: DevOps + Security.
- Ressourcen: 1 Tag.
- Erfolgskriterien: Security-Befunde koennen Builds technisch stoppen.

Welle 3 — bis 2026-07-10:

- GAP-07
- Schritte: 2FA nur fuer einen formal designten Live-Pfad implementieren; fehlendes Secret muss fail-closed sein.

- Eigentuermer: Security + Backend.
- Ressourcen: 1 Tag.
- Erfolgskriterien: ein fehlendes oder ungultiges TOTP-Setup blockiert jede Live-Aktivierung.
- GAP-08
- Schritte: Certificate Pinning entweder korrekt konfigurieren und aktiv erzwingen oder das Modul bis zur echten Einfuehrung als nicht aktiv deklarieren.
- Eigentuermer: Security.
- Ressourcen: 1 Tag.
- Erfolgskriterien: keine „Scheinkontrolle“ mehr im Default-Zustand.
- GAP-09
- Schritte: Task-, Bridge- und WebSocket-Stubs mit klaren Feature-Flags, Roadmap-Hinweisen oder bewusstem Rueckbau versehen.
- Eigentuermer: Produkt + Backend.
- Ressourcen: 1 Tag.
- Erfolgskriterien: Scope und Reifegrad jedes Nebenpfads sind fuer Entwickler und Betreiber eindeutig.

9. Schlussfolgerung und Bereitschaftsbestimmung

Bereitschaftsstatement:

- Ja fuer lokalen und CI-gestuetzten dry_run-Entwicklungsbetrieb.
- Nein fuer unbeaufsichtigten, containerisierten Dauerbetrieb gemaess aktueller Betriebsdoku.
- Nein fuer jede spaetere Live-Aktivierung, solange GAP-01, GAP-02, GAP-03, GAP-05 und GAP-06 nicht geschlossen sind.

Kritische Blocker:

- Kein aktiver Health-/Metrics-Pfad trotz dokumentierter Erwartung.
- Fehlverdrahteter Monitoring-/Alerting-Stack.
- Inkonsistente Build-/Dependency-Artefakte.
- Nicht genutztes Default-Rate-Limiting.
- Kontrolllandschaft teilweise nur auf Testebene vorhanden, nicht im Produktionspfad.

Gesamtfazit:

ai4trade-bot ist kein unreifes Projekt; es ist vielmehr ein technisches Projekt mit guter Entwicklungsbasis und erkennbarer Designabsicht, dessen Betriebsreife hinter seiner Implementierungstiefe zurueckbleibt. Die naechsten Schritte sind daher kein kompletter Neuaufbau, sondern gezielte Aktivierung, Konsolidierung und Verifikation bereits vorhandener Bausteine.

10. Anhaenge

Anhang A — Rohdaten und Evidenzartefakte

- Volltestlauf am 2026-05-29: 320 passed in 57.19s.

- Workspace-Analyse: get_errors meldete keine aktuellen Fehler.
- Wichtige geprüfte Dateien:
- main.py
- config.py
- pyproject.toml
- .github/workflows/ci.yml
- Dockerfile
- docker-compose.yml
- monitoring/prometheus.yml
- monitoring/alertmanager_rules.yml
- adapters/signal_publisher.py
- adapters/task_handler.py
- adapters/rate_limiter.py
- core/health.py
- core/metrics.py
- core/secret_provider.py
- core/two_factor.py
- core/ssl_context.py
- exchanges/factory.py
- exchanges/bitget_rest.py
- exchanges/market_stream.py
- storage/sqlite_repository.py
- docs/operations.md
- docs/incident-response.md
- docs/recovery.md

Anhang B — Interviewnotizen

- Keine Interviews durchgeführt.
- Keine Betriebs- oder Management-Statements ausserhalb des Repository-Kontexts ausgewertet.

Anhang C — Annahmen und fehlende Daten

- Es lagen keine Produktionsmetriken, Restore-Drills oder On-Call-Nachweise vor.
- Die tatsächliche Verfügbarkeit von curl und sqlite3 im finalen Containerimage wurde nicht ausserhalb des Repositories verifiziert.
- Es wurde keine rechtliche Bewertung fuer Live-Trading oder externe Regulierung vorgenommen.

Anhang D — Glossar

- dry_run: Simulationsmodus ohne Live-Handelsaktivierung.
- WAL: Write-Ahead Logging fuer SQLite.

- Observability: Beobachtbarkeit ueber Metriken, Logs und Healthsignale.
- Rate Limiting: technische Begrenzung von Request-Raten gegen externe APIs.
- Audit-Trail: nachvollziehbare Spur relevanter Systementscheidungen und Ereignisse.
- Fail-open: Kontrolle laesst bei Fehlern oder fehlender Konfiguration dennoch passieren.
- Fail-closed: Kontrolle blockiert bei Fehlern oder fehlender Konfiguration.